SCHEDULING: PART 2

Nineteenth Lecture by Prof. Uday S. Dixit, Department of Mechanical Engineering, IIT Guwahati



Critical Ratio

- For organizations and industries that have due dates, the critical ratio for sequencing jobs is beneficial.
- The **critical ratio** (C_R) is an index number computed by dividing the time remaining until due date by the work time remaining. Critical ratio performs better than FCFS, SPT, EDD, or LPT on the average job lateness criterion.
- The critical ratio gives priority to jobs that must be done to keep shipping on schedule.
  - A job with a low critical ratio (less than 1.0) is one that is falling behind schedule. If C_R is exactly 1.0, the job is on schedule.
  - C_R greater than 1.0 means the job is ahead of schedule and has some slack.

Critical Ratio

$$C_R = \frac{\text{Time remaining}}{\text{Workdays remaining}} = \frac{\text{Due date} - \text{Today's date}}{\text{Work (lead) time remaining}}$$

- ***Example:*** Suppose today is day 2 for an industry's production schedule. Three jobs are on order as shown in Table.

Job	Due date	Workdays remaining
A	13	10
B	5	4
C	9	7

Job	Critical ratio	Priority order
A	(13–2)/10 = 1.1	3
B	(5–2)/4 = 0.75	1
C	(9–2)/7 = 1	2

Job B has a critical ratio of less than 1, meaning it will be late unless expedited. Thus, it has the highest priority.

Job C is on time, and job A has some slack. Once job B is completed, the critical ratios for jobs A and C to determine whether their priorities change or not.

Critical Ratio

- In scheduling systems, the critical-ratio rule can help in following ways:
  - Determine the status of a specific job.
  - Establish relative priority among jobs on a common basis.
  - Adjust priorities (and revise schedules) automatically for changes in both demand and job progress.
  - Dynamically track job progress.

Sequencing Problems

- Problem of preparing optimal timetable to support the production schedule.
- The most common objective: Minimization of the total elapsed time.
- Suppose *n* jobs are to be processed, one at a time, on each of *m* machines. Order of machines is known. Number of possible sequences: $(n!)^m$.
- Suppose there are 10 jobs to be processed on 4 machines.
- On one machine, there are 10! ways to assign the job.
- $10!=10\times9\times8\times7\times6\times5\times4\times3\times2\times1=3,628,800$
- But there are four machines. Hence, total ways:

$$3,628,800\times3,628,800\times3,628,800\times3,628,800\cong1.734\times10^{26}$$

Sequencing *N* jobs on two machines: Johnson's Rule

- **Johnson's Rule:** It minimizes the total time for sequencing a group of jobs through two work centers while minimizing total idle time in the work centers.
- This is the case where *N* jobs (where $N \geq 2$) go through two different machines or work centers in the same order. (Each work center only works on one job at a time)
- ***Johnson's rule* involves the following steps:**
  - Find the smallest processing time on Machine 1 and Machine 2.
  - a) If the smallest value is in Machine 1 process that job first.
    b) If the smallest value is in the Machine 2 process that job last.
  - a) If the minimum time of a job for both Machine 1 and Machine 2 is equal, then perform the job of Machine 1 at first.

    b) If there is a tie for a minimum time among jobs in Machine 1, select the job corresponding to the minimum of Machine 2 and process it first.

Sequencing *N* jobs on two machines: Johnson's Rule

- c) If there is a tie for a minimum time among jobs in Machine 2, select the job corresponding to the minimum of Machine 1 and process it last.
- Calculate the total elapsed time(the time interval between starting the first job and completing the last job) and Idle time(the time during which the machine remains idle during the total elapsed time.

https://www.geeksforgeeks.org/johnsons-rule-in-sequencing-problems/

Sequencing *N* jobs on two machines: Johnson's Rule

- ***Example:*** **Five different jobs are to be processed through two different machines. The time (in hours) for processing each job is shown in Table. Set the sequence to minimize the total time of six jobs.**

- ***Solution*:**

The job with shortest time is 2 (Job A) in machine 1. Hence, it is processed last.

A				

Next, the shortest time is 3 (Job B) for machine 2, process it last.

A				B

Next, the shortest time is 4 (Job C) for machine 1, process it first

A	C			B

Job	Machine 1	Machine 2
A	2	5
B	6	3
C	4	9
D	7	12
E	10	5

Sequencing *N* jobs on two machines: Johnson's Rule

- ***Example:*** **Five different jobs are to be processed through two different machines. The time for processing each job is shown in Table. Set the sequence to minimize the total time of six jobs.**

Job	Machine 1	Machine 2
A	2	5
B	6	3
C	4	9
D	7	12
E	10	5

- ***Solution*:**

Next, the shortest time is 5 (Job A and E) for machine 2.

However, Job A is already assigned. Hence, assign Job E from first in the sequence.

A	C		E	B

Then, Job D is assigned.

A	C	D	E	B

Sequencing *N* jobs on two machines: Johnson's Rule

- ***Solution***: The final sequence of processing the jobs is **A, C, D, E, B**

Job	Machine 1		Machine 2	
	In time	Out time	In time	Out time
A	0	0+2=2	2	2+5=7
C	2	2+4=6	7	7+9=16
D	6	6+7=13	16	16+12=28
E	13	13+10=23	28	28+5=33
B	23	23+6=29	33	33+3=36

Job	Machine 1	Machine 2
A	2	5
B	6	3
C	4	9
D	7	12
E	10	5

- Total elapsed time = 36 hours

https://www.geeksforgeeks.org/johnsons-rule-in-sequencing-problems/

A workshop has **five jobs** (J1, J2, J3, J4, J5) that need to be processed on **two machines**:

- **Machine A (first), then**
- **Machine B (second)**

The processing times (in minutes) for each job on both machines are given below:

Job	Machine A	Machine B
J1	5	9
J2	8	7
J3	6	4
J4	3	8
J5	7	6

Step 1: Find the smallest processing time:

- **3** (Job J4 on Machine A) → Schedule **J4 first**.

Sequence: [**J4**, _, _, _, _]

Remove J4 from the list.

Step 2: Remaining jobs:

Job	A	B
J1	5	9
J2	8	7
J3	6	4
J5	7	6

Smallest time: **4** (Job J3 on Machine B) → Schedule **J3 last**.

Sequence: [J4, _, _, _, **J3**]

Step 3: Remaining jobs:

Job	A	B
J1	5	9
J2	8	7
J5	7	6

Smallest time: **5** (J1 on A) → Schedule **J1 early**.

Sequence: [J4, **J1**, _, _, J3]

Step 4: Remaining jobs:

Job	A	B
J2	8	7
J5	7	6

Smallest time: **6** (J5 on B) → Schedule **J5 late** (before J3).

Sequence: [J4, J1, _, **J5**, J3]

Remaining: J2 → Goes in the only empty spot.

Final sequence:

→ [**J4, J1, J2, J5, J3**]

Table for Gantt Chart

Job	A Start	A Finish	B Start	B Finish
J4	0	3	3	11
J1	3	8	11	20
J2	8	16	20	27
J5	16	23	27	33
J3	23	29	33	37

Total Elapsed Time (Makespan) = Time when last job finishes on Machine B

→ 37 minutes